

Education

University of Illinois at Urbana-Champaign | M.S. in Computer Science, 2023/08 - 2024/12
University of Illinois at Urbana-Champaign | B.S. in Computer Engineering, GPA 4.0/4.0 2019/08 - 2023/05
– Honors: Highest Honors, Bronze Tablet (3%, 2023), Dean's List (2020 & 2022)

Coursework: ML for Compilers, AI Efficiency, ML Algorithms for LLMs, GPU Programming, Machine Learning, Artificial Intelligence, Computer Networks, Distributed Systems, Computer Systems, Digital systems, Database Systems.

Skills

Programming Languages: Python, C/C++, CUDA, SQL, SystemVerilog, Assembly

ML Framework & Compiler Technologies: PyTorch, Triton, XLA, VeRL, vLLM, MLIR, TVM, PyG

Systems & Cloud: AWS, GCP, Kubernetes, Docker, Performance Profiling & Optimization, Distributed Systems

Experience

AWS, Annapurna Labs | **Software Engineer** | *ML Compiler, LLMs, LLM Systems, Architecture* 2025/03 - Present

- Performance optimizations for LLMs workloads, implementing custom flash-attention kernels for various scenarios.
- Developing kernel languages ([NKI](#)) for AWS AI accelerators ([Trainium](#)) that is user-friendly, robust, and efficient.
- Developing [AWS Neuron](#), architecting and managing critical compiler optimization passes like scheduling and allocation to achieve optimal performance. Using ideas of *software pipelining*, *modulo allocation*, etc.
- Managing and advancing the internal autotuning infrastructure to systematically improve ML compiler performance.

AWS, Annapurna Labs | **Software Engineer Intern** | *ML Compiler, LLMs, LLM Systems* 2024/05 - 2024/08

- Delivered three organization-wide presentations and received a [Certificate of Appreciation](#) for developing the most influential internship projects, *significantly enhancing performance, and opening up new directions*.
- Developed pipeline for automatic kernel generation, compilation, profiling, and visualization with a design space.
- Collected data for DMA access pattern analysis and introduced a learning-based DMA latency model.
- Created the *first-generation Autotuning infrastructure* from scratch for compiler optimization and kernel optimization.
- Used autotuning to optimize the Matmul Fusion Pass, achieving a 14.7% improvement for the Llama3.1 model.
- Developed the kernel languages ([NKI](#)) to support the kernel optimization with autotuning, achieving a 4.9% HFU improvement for attention kernels.
- Implemented multi-process compilation and distributed profiling, resulting in 8.62X speedup for autotuning.

TikTok | **Software Engineer Intern** | *ML, Graphics, Game Engine, AR/VR* 2022/05 - 2022/08

- Collaborated in developing TikTok's 3D Game Engine, which empowers users to create/use interactive AR/VR stickers.
- Implement a query-based animation system Motion Matching in C++ for realistic and responsive avatar control.
- Developed an SDK for Skeleton Retargeting in C++ and Lua, supporting animation adaptation across character models, and integrated the Text-to-Animation algorithm into the engine using the SDK.

University of California, Los Angeles | **Research Assistant** | *ML, RL, GNN, FPGA, Compiler* 2022/06 - 2022/11

- Worked with Prof. Jason Cong on combining GNN-based cost model with an ML/RL-based Design Space Exploration to achieve FPGA Accelerator Design Automation.
- Developed a learning-based Cost Model with GNN as a surrogate of the HLS tool for quick and accurate profiling.
- Optimized DSE by deploying heuristic algorithms such as Genetic Algorithm and Simulated Annealing.
- Used Reinforcement-Learning and Bandits for automatic algorithm selection, boosting exploration speed by 11%.

Projects

Optimized GPU code generation framework for SParse reguLAR Attention 2023/08 - 2024/07

- Developed SPLAT, an optimized framework for efficient sparse-MHSA, targeting moderate sparsity levels.
- Introduced Affine Compressed Sparse-Row (ACSR) format for regular sparsity patterns in MHSA.
- Engineered advanced GPU code-generation algorithms for ACSR, enhancing sparse-MHSA kernel performance.
- Achieved 2.05x and 4.05x speedups over Triton and TVM kernels with SPLAT implementation.

Implement A “[Doodle Jump](#)” Efficiently on the FPGA Board 2022/01 - 2022/05

- Ported the game “Doodle Dump” to FPGA with SystemVerilog, achieving low power consumption and high efficiency.
- Implemented a SOC with NIOS II in C to manage complex tasks like USB protocol and memory I/O.
- Consumed only 400KB memory, 0.5w power to achieve a 50hz frame rate, won the *Best Design Prize*.

Publications

- [SPLAT](#): A framework for optimised GPU code-generation for SParse reguLAR ATtention (OOPSLA' 25)